

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab10-CUDA 并行矩阵乘法

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2025 年 6 月 13 日

1 实验目的

- 理解 CUDA 编程模型 (Grid、Block、Thread) 及其在矩阵乘法中的应用。
- 学习 GPU 内存优化技术。

2 实验内容

- 实现基础矩阵乘法
- 优化矩阵乘法：共享内存，分块技术
- 测量不同实现的运行时间

3 实验原理

3.1 CUDA 编程模型与矩阵乘法

CUDA 编程模型将计算任务组织为 Grid-Block-Thread 三级层次结构。在矩阵乘法 $C = A \times B$（其中 A 为 $M \times K$ 矩阵，B 为 $K \times N$ 矩阵，C 为 $M \times N$ 矩阵）中，每个输出元素 $C_{ij} = \sum_{k=0}^{K-1} A_{ik} \cdot B_{kj}$ 的计算是独立的，天然适合并行化。

实验采用三种不同粒度的优化策略：

1. **朴素实现（Naive）**：每个线程直接从全局内存读取数据，计算一个输出元素。全局内存访问延迟通常为数百个时钟周期，且存在大量重复读取——A 的同一行被 N 个线程重复读取，B 的同一列被 M 个线程重复读取。
2. **共享内存分块（Shared Memory Tiling）**：将矩阵划分为子块（tile），每次将一个 tile 的数据从全局内存批量加载到共享内存（on-chip，延迟约为 20–30 个时钟周期），块内所有线程可重复使用这些数据，实现” 一次加载，多次计算”。
3. **寄存器分块（Register Tiling）**：在共享内存分块基础上，每个线程进一步使用寄存器缓存一个小型子块（如 4×4），在寄存器中完成多次乘累加后再写回，将计算访存比从 0.5 提升至 2.0（以 tile_size=4 为例）。

3.2 数据划分策略

实验实现了三种数据/任务划分方式，分别对应不同的 Grid 和 Block 组织方式：

- **按行划分（Row-wise）**：Grid 为 1 维，每个 thread block 负责一组完整的行（所有 N 列）。Block 内线程沿列方向并行，沿 K 方向分块迭代。此方式对 A 矩阵访问具有良好的空间局部性（行优先存储），但对 B 矩阵的列访问需要跨步读取。

- **按列划分（Column-wise）**：Grid 为 1 维，每个 thread block 负责一组完整的列（所有 M 行）。此方式对 B 矩阵访问有良好的空间局部性，但 A 矩阵需要按行读取（恰好与行优先存储匹配），而 B 的列访问存在严重的非合并访存问题。
- **按数据块划分（Block-wise / 2D Tiling）**：Grid 为 2 维，每个 thread block 负责 C 矩阵的一个 2 维子块。这是最标准且最高效的 CUDA 矩阵乘法实现方式，兼顾了 A 和 B 的访存合并性，同时提供了两个维度的并行度。

4 实验实现

- 实现基础 CUDA 矩阵乘法（朴素全局内存版本）
- 优化矩阵乘法：基于共享内存的分块优化
- 进一步优化：基于寄存器的分块优化（寄存器复用）
- 实现三种数据划分策略：按行划分、按列划分、按数据块划分
- 测试不同矩阵规模（512、1024、2048）和线程块大小（8×8、16×16、32×32）
- 测量并对比不同实现的运行时间，分析性能差异原因

4.1 朴素实现——按数据块划分

```
__global__ void matmul_naive_block(const float *__restrict__ A,
                                   const float *__restrict__ B,
                                   float *__restrict__ C,
                                   int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; k++) {
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

Listing 1: 朴素矩阵乘法（全局内存，2D Grid + 2D Block）

朴素实现中，每个线程计算一个输出元素，遍历整个 K 维度。主要问题是：（1）每个线程都需要从全局内存读取 $2K$ 个浮点数，访存量极大；（2）A 的同一行被 N 个线程重复读取，B 的同一列被 M 个线程重复读取，全局内存带宽被严重浪费。

4.2 共享内存分块实现

```cpp
template <int BLOCK_SIZE>
__global__ void matmul_shared_block(const float *__restrict__ A,
                                    const float *__restrict__ B,
                                    float *__restrict__ C,
                                    int M, int N, int K) {
    __shared__ float As[BLOCK_SIZE][BLOCK_SIZE];
    __shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE];

    int row = blockIdx.y * BLOCK_SIZE + threadIdx.y;
    int col = blockIdx.x * BLOCK_SIZE + threadIdx.x;

    float sum = 0.0f;
    for (int tk = 0; tk < K; tk += BLOCK_SIZE) {
        // 协作加载A的子块到共享内存
        if (row < M && (tk + threadIdx.x) < K)
            As[threadIdx.y][threadIdx.x] = A[row * K + tk + threadIdx.x];
        else
            As[threadIdx.y][threadIdx.x] = 0.0f;

        // 协作加载B的子块到共享内存
        if ((tk + threadIdx.y) < K && col < N)
            Bs[threadIdx.y][threadIdx.x] = B[(tk + threadIdx.y) * N + col];
        else
            Bs[threadIdx.y][threadIdx.x] = 0.0f;

        __syncthreads();  // 保证整个block的数据加载完成

        // 利用共享内存进行乘累加
        for (int k = 0; k < BLOCK_SIZE; k++) {
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        }
        __syncthreads();  // 保证下一轮加载前所有线程用完当前数据
    }

    if (row < M && col < N)
        C[row * N + col] = sum;
}
```

Listing 2: 共享内存分块矩阵乘法

共享内存分块的核心思想：将 K 维度切分为多个大小为 `BLOCK_SIZE` 的 tile。每个 thread block 协作将 A 和 B 的一个 tile 加载到共享内存中，然后所有线程使用共享内存中的数据进行计算。数据重用率显著提高——每个从全局内存加载的元素被 block 内

BLOCK_SIZE 个线程共享使用。

4.3 寄存器分块实现

```
// TILE_DIM = 32 / BLOCK_SIZE, 即每个线程计算 TILE_DIM×TILE_DIM 个元素
__shared__ float As[BLOCK_SIZE * TILE_DIM][BLOCK_SIZE];
__shared__ float Bs[BLOCK_SIZE][BLOCK_SIZE * TILE_DIM];

// 寄存器中的累加器
float c[TILE_DIM][TILE_DIM] = {{0.0f}};
float a_reg[TILE_DIM], b_reg[TILE_DIM];

for (int tk = 0; tk < K; tk += BLOCK_SIZE) {
    // 协作加载: 每个线程加载TILE_DIM个元素到更大的共享内存tile
    for (int i = 0; i < TILE_DIM; i++) {
        As[threadIdx.y * TILE_DIM + i][threadIdx.x] = ...;
        Bs[threadIdx.y][threadIdx.x * TILE_DIM + i] = ...;
    }
    __syncthreads();

    // 利用寄存器进行高密度计算
    for (int k = 0; k < BLOCK_SIZE; k++) {
        for (int i = 0; i < TILE_DIM; i++)
            a_reg[i] = As[threadIdx.y * TILE_DIM + i][k];
        for (int j = 0; j < TILE_DIM; j++)
            b_reg[j] = Bs[k][threadIdx.x * TILE_DIM + j];
        // 外积累加: TILE_DIM^2次乘加, 仅2*TILE_DIM次寄存器加载
        for (int i = 0; i < TILE_DIM; i++)
            for (int j = 0; j < TILE_DIM; j++)
                c[i][j] += a_reg[i] * b_reg[j];
    }
    __syncthreads();
}
```

Listing 3: 寄存器分块矩阵乘法（关键计算部分）

寄存器分块的关键优化：每个线程处理的输出元素从 1 个增加到 TILE_DIM×TILE_DIM 个。在每次从共享内存加载 $2 \times$ TILE_DIM 个值到寄存器后，可完成 TILE_DIM^2 次乘加运算。以 TILE_DIM=4 为例，计算访存比从 0.5（朴素/共享内存）提升至 $16/(4+4) = 2.0$，提升 4 倍。设计中保持 BLOCK_SIZE×TILE_DIM=32 不变，使共享内存 tile 大小恒定为 32×32（4KB），确保在不同 block 大小下都有合理的共享内存使用量。

5 实验结果与分析

5.1 性能测试数据

实验在 NVIDIA Tesla V100S GPU 上进行，矩阵规模 $M = N = K \in \{512, 1024, 2048\}$，线程块大小为 8×8、16×16 和 32×32。实验具体流程如下：（1）使用 cuRAND 库在 GPU 端生成服从均匀分布 $U(0, 1)$ 的随机矩阵 A 和 B，固定随机种子以保证可复现性；（2）分别调用 9 种不同 kernel（3 种实现 $\times 3$ 种划分策略）执行矩阵乘法，每种配置先执行 1 次 warm-up 以消除冷启动偏差，再执行 5 次正式运行取平均时间；（3）将 GPU 计算结果传回主机，与 CPU 端串行矩阵乘法结果逐元素比较，验证条件为相对误差 $< 10^{-3}$ 或绝对误差 $< 10^{-3}$（对接近零值的情形）。所有 81 组测试配置均通过正确性验证。运行时间单位为毫秒（ms）。

表 1: 按行划分：性能对比 (时间单位: ms)

矩阵规模 (N)	线程块大小	朴素实现	基于共享内存优化	基于寄存器分块优化
512	8×8	1.292	0.728	1.326
	16×16	0.667	0.395	0.435
	32×32	0.534	0.352	0.359
1024	8×8	7.659	5.995	14.028
	16×16	10.189	4.754	4.935
	32×32	5.151	1.669	1.640
2048	8×8	45.625	20.068	33.432
	16×16	21.825	10.679	11.502
	32×32	11.768	6.717	6.803

表 2: 按列划分：性能对比 (时间单位: ms)

矩阵规模 (N)	线程块大小	朴素实现	基于共享内存优化	基于寄存器分块优化
512	8×8	0.867	0.767	1.489
	16×16	1.422	0.813	1.097
	32×32	5.466	2.211	2.210
1024	8×8	10.765	6.283	13.339
	16×16	11.288	6.271	9.348
	32×32	46.211	16.714	16.717
2048	8×8	39.278	22.740	36.521
	16×16	45.516	23.227	21.404
	32×32	88.361	35.918	35.919

表 3: 按数据块划分：性能对比 (时间单位: ms)

矩阵规模 (N)	线程块大小	朴素实现	基于共享内存优化	基于寄存器分块优化
512	8×8	0.189	0.123	**0.053**
	16×16	0.129	0.080	**0.052**
	32×32	0.140	0.083	0.081
1024	8×8	1.342	0.910	**0.303**
	16×16	0.943	0.584	**0.307**
	32×32	0.891	0.549	0.523
2048	8×8	19.286	8.155	**2.893**
	16×16	8.134	4.804	**2.457**
	32×32	7.746	4.613	4.140

5.2 分析性能差异的原因

- **结合 CUDA 内存模型和矩阵乘法原理，分析造成观察到的性能差异的可能原因。**

  回答：性能差异的根本原因在于 GPU 内存层次结构的带宽和延迟差异：

  1. **全局内存带宽瓶颈**：V100 的全局内存（HBM2）带宽约 900GB/s，但延迟高达数百周期。朴素实现中每个输出元素的每次乘加都需要 2 次全局内存读取（共 $2K$ 次），严重受限于内存带宽。由表3可见，N=2048 时朴素实现（block=16×16）耗时 8.134ms，而共享内存版本仅需 4.804ms，加速约 1.69×；N=1024 时朴素实现耗时 0.943ms，共享内存版本为 0.584ms，加速约 1.61×。这是因为朴素实现需读取约 $2N^3$ 个浮点数，例如 N=2048 时约 17.2×10^9 次全局内存读取（约 68.7GB），严重超出 L2 缓存容量，实际有效带宽远低于峰值。
  2. **共享内存减少全局访存**：共享内存带宽约 13.8TB/s（V100），延迟约 20–30 周期。通过分块将数据预加载到共享内存，每个 block 内的 `BLOCK_SIZE` 个线程共享同一份数据，全局访存减少约 `BLOCK_SIZE` 倍。从表3可以看出共享内存优化的显著效果：N=2048、block=32×32 时，朴素实现 7.746ms 降至共享内存的 4.613ms（加速 1.68×）；N=1024 时，相同配置下从 0.891ms 降至 0.549ms（加速 1.62×）。同时，共享内存的 bank 结构支持高效的并行访问。
  3. **寄存器进一步提升计算密度**：寄存器延迟仅 1 个周期，是最快的存储层次。寄存器分块将多次共享内存读取合并为一次寄存器加载，增加每字节加载对应的计算量（计算访存比），使 compute bound 的潜力得以发挥。表3的数据充分说明了这一点：N=2048、block=16×16 时，寄存器分块仅需 2.457ms，相比共享内存的 4.804ms 加速 1.96×，相比朴素实现的 8.134ms 加速 3.31×。对于

block=8×8（TILE_DIM=4），寄存器分块（2.893ms）相比共享内存（8.155ms）加速达 2.82×，体现了高计算访存比带来的显著收益。

4. **访存合并性**：按列划分性能极差的原因是线程束（warp）内的 32 个线程访问的是步长很大的非连续地址（行优先存储下，同一列的元素在内存中相距 K 个元素），无法合并为一个内存事务，导致实际有效带宽远低于峰值。对比表1和表2可知：N=2048、block=32×32 时，按行划分共享内存版本耗时为 6.717ms，而按列划分为 35.918ms，性能相差 5.35 倍；N=2048、block=16×16 时差异更明显（10.679ms vs 23.227ms，相差 2.17 倍）。此外，按列划分中 block=32×32 反而比 16×16 更慢（朴素实现：88.361ms vs 45.516ms），这与常规预期相反，原因是大 block 加剧了非合并访存的延迟惩罚。

- **如何选择合适的线程块大小以提高占用率?**

  回答：选择线程块大小的基本原则如下：

  1. **线程数应为 warp 大小（32）的整数倍**：确保 warp 充分利用，避免线程浪费。本实验使用的 8×8=64、16×16=256、32×32=1024 均满足此要求。
  2. **考虑 SM 资源限制**：V100 每个 SM 最多驻留 2048 个线程、64 个 warp、以及最多 32 个线程块。线程块过小（如 8×8=64 线程）会导致 SM 上线程总数受限于 block 数量上限（32 blocks × 64 threads = 2048 threads，刚好达到上限），但此时每个线程的寄存器使用量是限制因素。线程块过大（如 32×32=1024 线程）则每个 SM 只能驻留 2 个 block（2048/1024=2），限制了延迟隐藏能力。
  3. **中等大小的线程块（128–256 线程）通常最优**：在本实验中，16×16 线程块通常能够在占用率、共享内存利用率和访存效率之间取得较好的平衡，因此整体表现较为稳定。实验结果表明，16×16 线程块在大多数测试中具有较好的综合性能，但并非所有情况下都是最优。实际性能还受到寄存器使用量、共享内存占用率以及矩阵规模等因素影响。例如，N=2048、按数据块划分时，16×16 的寄存器分块（2.457ms）优于 32×32（4.140ms），但在共享内存版本中 32×32（4.613ms）略优于 16×16（4.804ms），说明不同优化策略下的最优 block 大小并不相同。
  4. **结合寄存器分块时需重新权衡**：寄存器分块增加了每线程的寄存器使用量，降低了 SM 上可驻留的 block 数。此时较小线程块（如 8×8）配合较大的 TILE_DIM（如 4），可能获得更好的计算访存比，弥补线程数不足的劣势。

- **思考如果按不同的方式划分（例如，按行、列、数据块划分），可能会对性能和实现复杂度带来什么影响?**

  回答：

表 4: 三种划分策略对比总结

特性	按行划分	按列划分	按数据块划分
Grid 维度	1D	1D	2D
并行度	有限（= 行数/block_size）	有限（= 列数/block_size）	充分（≈ N^2/tile_size^2）
A 矩阵访存	合并性好	合并性好	合并性好
B 矩阵访存	较好	非合并，性能差	合并性好
实现复杂度	低	低	高
共享内存效率	较高	较低	最高
相对性能（N=2048）	1×（基线）	0.3–0.5×	2–3×

  - **按行划分**：实现相对简单（1D grid），对 A 矩阵访问合并性好（每行连续存储），但当 M 较小时并行度不足。适合 $M \gg N$ 的瘦高矩阵乘法场景。
  - **按列划分**：同样实现简单（1D grid），但由于矩阵在内存中按行优先存储，访问 B 矩阵的一列意味着跨步读取（stride = N），导致非合并访存，严重降低性能。除非配合矩阵转置等预处理，否则不建议使用。
  - **按数据块划分**：实现略微复杂（需要 2D grid 和更精细的索引计算），但提供了最佳的并行度（$O(N^2/\text{tile_size}^2)$ 个线程块）、最好的访存合并性（两个矩阵的 tile 访问都连续），以及最高的共享内存利用率。是通用矩阵乘法的标准实现方式，也是 cuBLAS 等库的底层策略。

- **何时应该优先考虑使用哪种存储?**

  回答:

  GPU 存储层次的选择应遵循" 尽可能使用更快、更靠近计算单元的存储" 原则:

  1. **全局内存（Global Memory）**：仅在数据首次加载和最终写回时使用。应尽量减少运行时的全局内存访问。适合存储整个矩阵，但计算中应避免频繁读取。
  2. **共享内存（Shared Memory）**：当同一个 block 内的多个线程需要重复使用相同数据时优先使用。在矩阵乘法中，A 的每一行和 B 的每一列都被多个线程共享，因此使用共享内存缓存 tile 极为有效。使用共享内存的典型场景: **(1)** 数据存在线程间复用; **(2)** 访问模式需要重排以改善全局内存合并性; **(3)** 需要线程间通信（如规约操作）。
  3. **寄存器（Register）**：当单个线程需要反复使用少量数据时优先使用。寄存器数量有限（V100 每线程最多 255 个），应分配给最频繁访问的变量。在矩阵乘法中，累加器 `c` 和从共享内存预取的 `a_reg`、`b_reg` 天然适合放在寄存器

中。寄存器分块通过增加每线程的寄存器用量来换取更高的计算访存比，在算术强度足够高时效果显著。

4. **选择策略**：从外到内，数据应逐步从全局内存迁移到共享内存再到寄存器。每一步迁移都应带来数据重用或访存模式改善。对于矩阵乘法：**先使用共享内存**消除冗余的全局访存；**再使用寄存器**进一步提高计算密度。但需注意——寄存器是稀缺资源，过度使用会降低占用率，需要在计算密度和并行度之间权衡。